



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Lab Report No:- 7

Title:- FIFO page Replacement Algorithm.

Submitted by:-

Name:- Surajan Shrestha

Roll No:- 38

Submitted To:- Drish mali

Department Of CSIT

Checked by:-

Date of submission:- 2081/05/28

TITLE : FIFO Page Replacement Algorithm

OBJECTIVE :

- To implement and simulate the FIFO page replacement algorithm.
- To calculate the number of page faults and page hits.
- To observe the efficiency of the FIFO algorithm by calculating the page fault rate and page hit rate.

THEORY :

→ In a computer system, memory management plays a crucial role in optimizing performance. Virtual memory allows processes to execute even if they are not entirely in main memory. A page replacement algorithm comes into play when a requested page is not in memory and a replacement is needed. The FIFO page replacement algorithm is one of the simplest algorithms used to manage memory. It replaces the oldest page in memory when a new page is required.

- **page** : A fixed-length contiguous block of virtual memory.
- **frame** : A fixed-size block of physical memory where pages are loaded.
- **Page fault** : Occurs when a requested page is not in memory and system must fetch it from disk.

- Page Hit: Occurs when the requested page is already present in memory.

The FIFO algorithm maintains a queue where pages are loaded in the order they arrive. When a page fault occurs, the oldest page is replaced by the new page.

OBSERVATION :

// source code

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int search (int frames[], int n, int page)
```

```
{ for (int i = 0; i < n; i++)  
  { if (frames[i] == page)
```

```
    { return i; } }
```

```
return -1;
```

```
}
```

```
void printframes (int frames[], int n)
```

```
{ printf (" | ");
```

```
for (int i = 0; i < n; i++)
```

```
  { if (frames[i] == -1)
```

```
    { printf (" | ");
```

```
    }
```

```
  else
```

```
  { printf ("%d |", frames[i]);
```

```
    }  
    printf("\n");  
}
```

```
void jiffo(int pages[], int n, int capacity)
```

```
{ int frames[capacity];
```

```
  int faults = 0;
```

```
  int nextframe = 0;
```

```
  for (int i = 0; i < capacity; i++)
```

```
  { frames[i] = -1;
```

```
  }
```

```
  printf("Page Replacement process :\n");
```

```
  printf("-----\n");
```

```
  for (int i = 0; i < n; i++)
```

```
  { int page = pages[i];
```

```
    printf("page Requested : %d\n", page);
```

```
    if (search(frames, capacity, page) == -1)
```

```
    { frames[nextframe] = page;
```

```
      nextframe = (nextframe + 1) % capacity;
```

```
      faults++;
```

```
      printf("page fault : \n");
```

```
    }
```

```
  else
```

```
  { printf("page found in frame : \n");
```

```
  }
```

```
  printfframes(frames, capacity);
```

```

printf("-----\n");
}
printf("\n");
printf("Total page faults : %d\n", faults);
printf("page fault Rate : %.3f\n", (float) faults/n);
printf("\n");
printf("Total page hits : %d\n", n-faults);
printf("page Hit Rate : %.3f\n", (float) (n-faults)
/n);

```

```

}
int main()
{
    int n;
    int capacity;
    printf("Enter the number of page requests:");
    scanf("%d", &n);
    if (n <= 0)
    {
        printf("Error : Invalid number of page requests\n");
        return 0;
    }
}

```

```

int pages[n];
printf("Enter the page requests :");
for (int i = 0; i < n; i++)
{
    scanf("%d", &pages[i]);
}
printf("Enter the capacity of the frame :");
scanf("%d", &capacity);
if (capacity <= 0)

```



```

{
    printf("Error : Invalid capacity.\n");
    return 0;
}
fifo(pages, n, capacity);
return 0;
}

```

OUTPUT:

Enter the number of page requests : 5
 Enter the page requests : 1 2 3 1 4
 Enter the capacity of the frame : 3
 Page Replacement process:

Page Requested : 1
 Page fault !
 1 1 1 1

Page Requested : 2
 Page fault !
 1 2 1 1

Page Requested : 3
 Page fault !
 1 1 2 1 3 1

Page Requested : 1
 Page found in frame !
 1 1 2 1 3 1

Page Requested : 4

Page fault!

1 4 1 2 1 3 1

Total page faults : 4

Page fault Rate : 0.800

Total Page hits : 1

Page Hit Rate : 0.200

DISCUSSION:

The FIFO Page replacement algorithm has a straightforward approach: the page that was loaded into memory first is the one to be replaced when needed. This algorithm is simple to implement but can suffer from Belady's anomaly, where increasing the number of frames may result in more page faults. This happens because the algorithm does not consider the frequency or recency of page use, which can lead to suboptimal performance in some cases. In the context of system performance, FIFO might not always be the most efficient page replacement algorithm. More advanced algorithms like LRU & optimal page replacement are often used in practical systems as they account for page's usage patterns.

CONCLUSION:

→ The FIFO page replacement algorithm is simple to implement but can be inefficient as it doesn't account for how recently or frequently a page is used. It highlights basic memory management concepts and the trade offs between simplicity & performance. The experiment effectively demonstrated page faults, hits & provided metrics for analyzing performance.